

Documentation

IoT Trojan Detector

Spotting trojan-like behaviour on IoT devices from network flows: a CSV-driven detector, a safe Docker lab and a harmless trojan simulator.

Contents

1. Overview

2. How the core logic works

3. Architecture and data model

4. Screen walkthrough

5. Mobile view

6. Running locally

7. Testing

8. Deployment

9. Five-minute presentation script

Every screenshot has numbered callouts. The explanations under each picture use the same numbers, and both are generated from one definition file (`docs/build/screens.py`).

1. Overview

IoT cameras are inexpensive, rarely patched and often ship with default passwords, so they are attractive targets for trojans and botnets. A compromised camera keeps streaming video normally while it quietly beacons to a command-and-control (C2) server, scans its neighbours or uploads data.

IoT Trojan Detector looks for those behaviours in **network-flow records** (who talked to whom, on which port, how often, how many bytes). It needs no software on the device and no malware signatures, so it works on cameras that cannot be modified.

What is in the repository

Part	Purpose
<code>trojan-lab/detector</code>	Flask web app and API plus the pure-Python analysis engine (<code>engine.py</code>).
<code>trojan-lab/camera</code> , <code>rtsp</code>	Simulated cameras: weak-login admin page, background flows, and a looping RTSP test stream.
<code>trojan-lab/trojan_sim</code> , <code>c2</code>	A harmless trojan stand-in (beacon, scan burst, persistence files, EICAR test string) and a fake C2 listener.
<code>trojan-lab/sample_data</code>	Baseline, clean and infected flow CSVs for upload and tests.
<code>docs/build</code>	Scripts that regenerate the screenshots and this PDF.

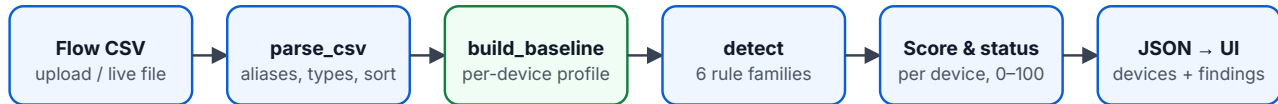
Safety. No real malware is used. The simulator only reproduces indicators, sends nothing but a short text beacon to a container on the same internal network, and the lab network has no route to the internet.

Key capabilities

- Upload any flow CSV (aliases and ISO/epoch timestamps accepted) with an optional known-good baseline.
- Nine detection rules across six families, each mapped to MITRE ATT&CK, with severity and a 0–100 risk score per device.
- One-click quarantine and release; a Docker lab to demonstrate infection and containment end to end.
- 15 automated tests; mobile-friendly UI.

2. How the core logic works

All analysis lives in `trojan-lab/detector/engine.py`, which uses only the Python standard library so it can be tested and reused without the web app.



baseline is optional: from a second upload, or frozen from the live capture

Step 1: parse and normalise

`parse_csv` matches column names case-insensitively against a canonical list and its aliases (e.g. `ts` → `timestamp`, `dport` → `dst_port`, `bytes` → `bytes_out`), converts timestamps (ISO-8601 or epoch seconds) to floats, skips malformed rows (aborting after 50), and sorts by time. A missing required column raises `FlowError` with a message naming the columns found, which the UI shows as the error banner.

Step 2: baseline (optional)

`build_baseline` summarises known-good traffic per device: the set of destination ports, destination IPs and (IP, port) pairs, plus the mean and standard deviation of flow size. This is what separates a legitimate heartbeat from a beacon: both are periodic, but only one goes to a peer the device has never used.

Step 3: the rules

Rule	Severity	Fires when	ATT&CK
<code>known_bad_ip</code>	critical	Any flow to an address on <code>KNOWN_BAD_IPS</code> (threat-intel list).	T1071
<code>suspicious_port</code>	high	Destination port in <code>SUSPICIOUS_PORTS</code> : 23, 2323, 4444, 6667, 1337, 31337, 9001, 12345.	T1571
<code>unusual_port</code>	medium	Port outside the device's baseline ports plus the default camera set (53, 80, 123, 443, 554, 1883, 3702, 5353, 8554, 8883). Suppressed when the port is already flagged above.	T1571
<code>new_external_dst</code>	medium	Baseline only: a non-LAN destination the device never talked to in the baseline.	T1071
<code>beaconing</code>	high	≥6 flows to one <code>dst:port</code> with inter-arrival jitter (CV) ≤0.25, size CV ≤0.5, mean gap ≥1 s, and the peer is <i>not</i> part of the baseline.	T1071 / T1029
<code>scan_hosts</code> / <code>scan_ports</code>	high	≥10 distinct destination hosts (or ports) inside any 60 s window.	T1018 / T1046
<code>exfil_volume</code>	high	≥5 MB sent to one non-LAN address.	T1041
<code>volume_outlier</code>	medium	Baseline only: a flow larger than baseline mean + 4σ and over 100 kB.	T1030

Why periodic traffic is not automatically a beacon

Camera video chunks, NTP and cloud heartbeats are all very regular. The beacon rule therefore only applies to a peer outside the device's baseline. With no baseline, regular traffic on well-known service ports (53, 80, 123, 443, 554...) is ignored unless the peer is on the known-bad list. Beaconing is measured by the *coefficient of variation* (standard deviation divided by mean) of the gaps between flows: a 15 s beacon with $\pm 4\%$ jitter scores about 0.02, far below the 0.25 limit, whereas human or application traffic is bursty and scores well above it.

Step 4: scoring

Each finding adds a weight: critical 40, high 25, medium 10, low 3. A device's score is the sum capped at 100. Status is **clean** below 10, **suspicious** from 10 and **compromised** from 50, so one critical plus one high finding (65) is enough for compromised, while a lone unusual port (10) only raises suspicion.

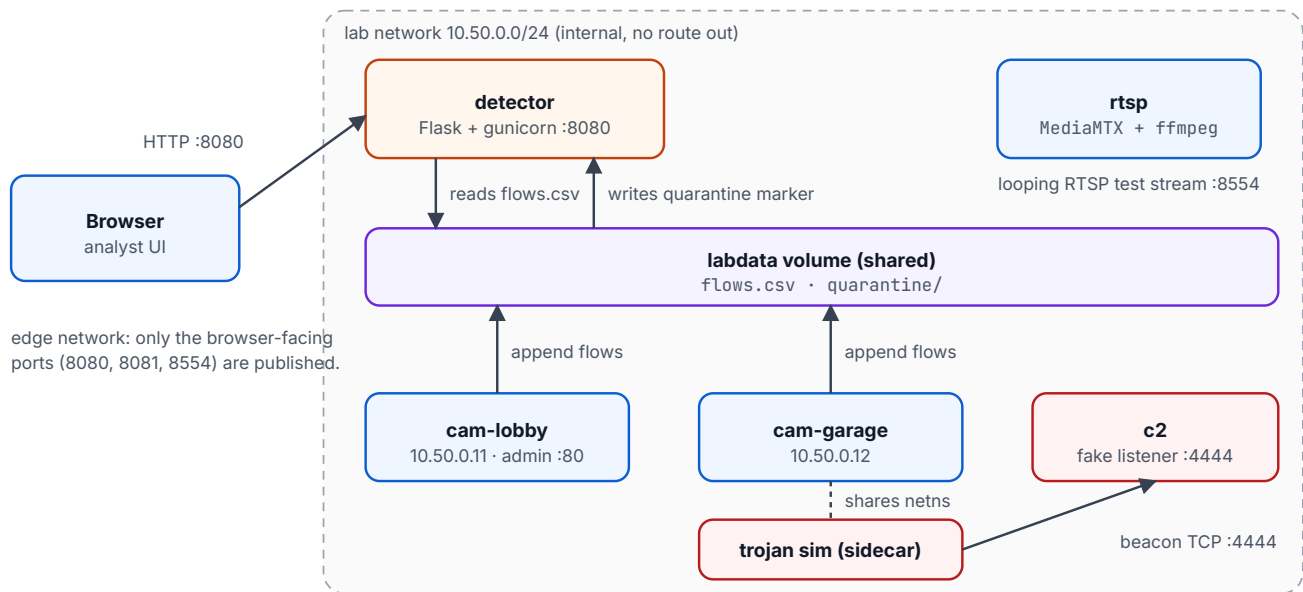
Worked example: the infected sample

Analysing `infected_flows.csv` against the baseline yields 22 findings on one device (2 critical, 7 high, 13 medium); the two other cameras stay at zero. One finding per rule:

Severity	Finding	Evidence (from the sample)
critical	Contact with known-bad IP	121 flow(s) to threat-intel listed address 203.0.113.66
high	Suspicious port 4444 (Metasploit default)	121 flow(s) to port 4444; destinations: 203.0.113.66
high	Periodic beaoning to 203.0.113.66:4444	121 flows, mean interval 15.0s (jitter CV 0.02), avg 310 B/flow
high	Scan-like burst across 10 distinct hosts	10 distinct hosts within 60s window
high	Scan-like burst across 10 distinct ports	10 distinct ports within 60s window
high	Large outbound transfer to 198.51.100.77	8.5 MB sent to external address
medium	Unusual destination port 22	1 flow(s) on port 22 outside the device's baseline profile
medium	New external destination 203.0.113.66	121 flow(s) to an address never seen in the baseline
medium	Flow size far above baseline	1 flow(s) exceed baseline mean+4 σ (4,302,583 B); max 8,500,000 B

The clean sample produces 0 findings and every device is clean.

3. Architecture and data model



Components

Service	Tech	Role
detector	Flask, gunicorn	Serves the UI and JSON API; runs the engine; writes quarantine markers.
cam-lobby, cam-garage	Python/Flask	Simulated cameras: admin page (<code>admin/admin</code> , deliberately weak) and normal flows appended to <code>flows.csv</code> .
rtsp	MediaMTX + ffmpeg	Looping test pattern at <code>rtsp://localhost:8554/cam</code> .
trojan	Python	Sidecar sharing cam-garage's network namespace, so beacons carry the camera's IP. Behind a Compose profile.
c2	Python sockets	Fake listener on :4444 that logs beacons and answers <code>ACK</code> .

Two Docker networks separate concerns: `lab` is `internal: true` (no route out), and `edge` only exists so a browser can reach the published ports. The simulated trojan lives only on `lab` . State is a shared volume and one in-memory baseline; there is no database.

Data model

Flow row (input CSV, one per flow)

Field	Type	Notes
<code>timestamp</code>	ISO-8601 or epoch	Required. Stored as float seconds (<code>ts</code>).
<code>device</code>	string	Required. Grouping key for all per-device logic.
<code>src_ip</code> , <code>dst_ip</code>	string	Required. LAN test uses RFC1918, loopback, link-local and multicast ranges.

<code>dst_port</code>	int	Required.
<code>protocol</code>	string	Required. Upper-cased (TCP, UDP...).
<code>bytes_out</code>	int	Required. Bytes sent by the device.
<code>bytes_in</code> , <code>packets</code> , <code>duration</code>	number	Optional; <code>bytes_in</code> is parsed, the others are accepted for compatibility.

Baseline profile (in memory, per device)

```
{ "cam-garage": { "ports": {554, 123, 53, 443}, "dsts": {...}, "pairs": {(ip, port), ...},
                  "flows": 396, "mean_bytes": 2.0e6, "std_bytes": 4.1e5 } }
```

Finding and device summary (API output)

```
finding = { device, rule, severity, title, detail, mitre,
            evidence: {dst_ip, dst_port, flows, interval_s, jitter_cv, ...}, first_seen,
            last_seen }
device  = { device, flows, bytes_out, src_ips[], score, status, findings }
result  = { summary: {flows, devices, findings, compromised, suspicious, with_baseline},
            devices[], findings[], quarantined[] }
```

HTTP API

Endpoint	Purpose
GET /	The single-page UI.
POST /api/analyze	Multipart <code>flows</code> (+ optional <code>baseline</code>); returns the result JSON. 400 with <code>{error}</code> on bad CSV, 413 above 25 MB.
POST /api/analyze/sample	JSON <code>{"name": "infected" "clean"}</code> ; runs a bundled sample against the bundled baseline.
POST /api/analyze/live	Analyses <code>\$LAB_DATA_DIR/flows.csv</code> .
POST /api/baseline/live	Freezes the live capture as the baseline.
POST DELETE /api/quarantine/<device>	Create or remove the marker file (name must match <code>[A-Za-z0-9_-]{1,64}</code>).
GET /health	Liveness check.

4. Screen walkthrough

The app is one page whose state changes as you work. These are all the states, in the order a demo visits them. Red numbered boxes in each screenshot match the numbered explanations beneath it.

Upload screen (landing page)

The detector is a single page. It opens on the upload card, where every analysis starts.

The screenshot shows the '1 · Upload flows' section of the application. It contains two file upload areas: 'Capture to analyse (required)' and 'Known-good baseline (optional)'. Each has a 'Choose File' button and a 'No file chosen' status. A blue 'Analyse' button is positioned to the right of the second upload area. Below these is a text area for 'required columns' with a list of fields: timestamp, device, src_ip, dst_ip, dst_port, protocol, bytes_out, bytes_in, packets, duration. Below the text area are four buttons: 'Load sample: infected', 'Load sample: clean', 'Analyse live lab capture', and 'Freeze live capture as baseline'. Red numbered boxes (1-7) are placed over various elements: 1 on the first 'Choose File' button, 2 on the second 'Choose File' button, 3 on the 'Analyse' button, 4 on the 'required columns' text area, 5 on the 'Load sample: infected' button, 6 on the 'Analyse live lab capture' button, and 7 on the 'Freeze live capture as baseline' button.

Figure: Upload screen (landing page)

- 1 **Capture to analyse (required).** The flow CSV to inspect: one row per network flow, exported from Zeek, Suricata, a router or the lab simulators.
- 2 **Known-good baseline (optional).** A CSV of normal traffic. It teaches the engine each device's usual ports and destinations; without it only built-in rules apply.
- 3 **Analyse.** Uploads both files to `POST /api/analyze` and renders the result below the card.
- 4 **Schema hint.** The seven required columns are generated from `engine.REQUIRED`; common aliases and ISO or epoch timestamps are also accepted.
- 5 **Load sample: infected / clean.** Runs a bundled capture against the bundled baseline, so you can try the tool with no files.
- 6 **Analyse live lab capture.** Reads `flows.csv` written by the Docker lab's simulated cameras and trojan.
- 7 **Freeze live capture as baseline.** Declares the traffic seen so far as "known good"; used in step 2 of the demo, before the trojan starts.

Files selected, ready to analyse

Choosing files only fills the form; nothing is sent until **Analyse** is pressed. Here the infected sample capture and the baseline are chosen.

The screenshot shows a web form titled "1 · Upload flows". It has two sections: "Capture to analyse (required)" and "Known-good baseline (optional)". In the first section, a file "infected_flows.csv" is selected, indicated by a red box and a red circle with the number 1. In the second section, a file "baseline_flows.csv" is selected, indicated by a red box and a red circle with the number 2. A blue button labeled "Analyse" is highlighted with a red box and a red circle with the number 3. Below the file selection area, there is a list of required columns: timestamp, device, src_ip, dst_ip, dst_port, protocol, bytes_out. Optional columns are bytes_in, packets, duration. Common aliases (ts, device_id, dport, proto, bytes...) are accepted; timestamps may be ISO-8601 or epoch seconds. At the bottom, there are four buttons: "Load sample: infected", "Load sample: clean", "Analyse live lab capture", and "Freeze live capture as baseline".

Figure: Files selected, ready to analyse

- 1 The capture file is attached (`infected_flows.csv`).
- 2 The baseline file is attached (`baseline_flows.csv`). Leave this empty to reuse the last baseline or run rules only.
- 3 Press **Analyse** to send both files. Uploads are limited to 25 MB.

Validation error

A CSV that lacks required columns is rejected with a specific, readable message instead of a stack trace.

The screenshot shows the same web form as before, but with a validation error. The "Capture to analyse (required)" section shows a file "my_flows.csv" selected, indicated by a red box and a red circle with the number 2. The "Known-good baseline (optional)" section shows "No file chosen". The "Analyse" button is still present. Below the file selection area, there is a red error banner with the message: "missing required column(s): src_ip, dst_port, protocol. Found: time, host, dst, bytes". This message is indicated by a red circle with the number 1. The rest of the form, including the column lists and bottom buttons, remains the same.

Figure: Validation error

- 1 **Error banner.** Names the missing column(s) and lists the columns that were found, so the CSV can be fixed quickly. The server returns HTTP 400 with `{"error": "..."}`.
- 2 The offending file is still selected; fix it and press Analyse again.

Results: summary tiles and device table

After analysis the page shows an at-a-glance summary and one row per device. The infected sample contains one trojan-like device among three cameras.

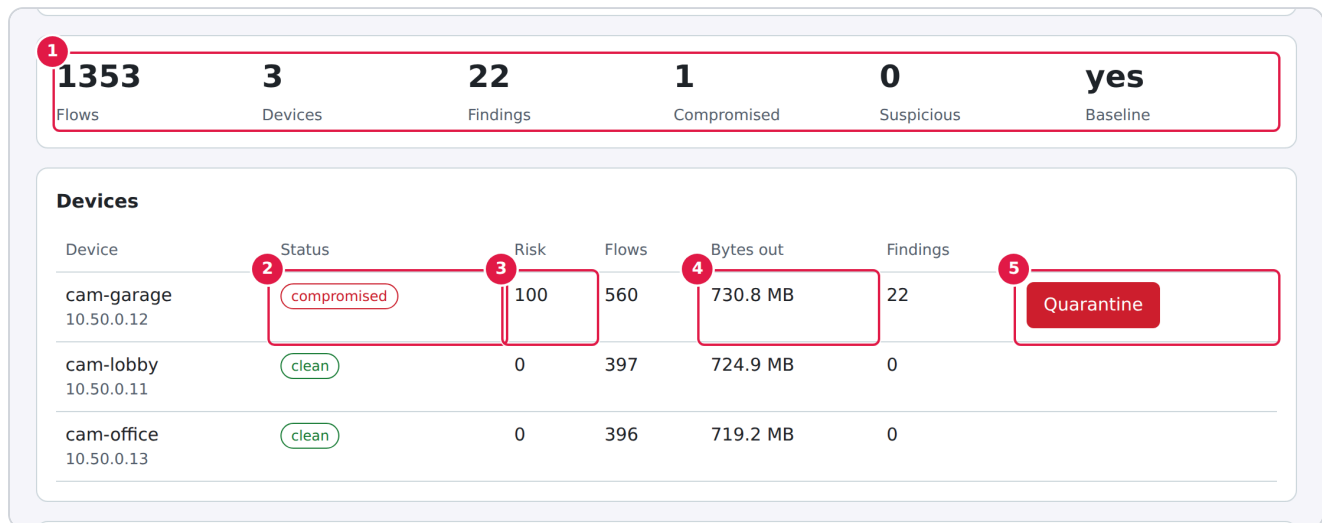


Figure: Results: summary tiles and device table

- 1 Summary tiles.** Flows analysed, devices seen, total findings, how many devices are compromised or suspicious, and whether a baseline was used.
- 2 Status pill.** *clean* (score < 10), *suspicious* (10–49) or *compromised* (50+). Rows are sorted riskiest first.
- 3 Risk score** (0–100): the sum of finding weights (critical 40, high 25, medium 10, low 3), capped at 100.
- 4 Bytes out.** Total data the device sent; a quick sanity check next to the exfiltration findings.
- 5 Quarantine.** Response action, shown only for devices that are not clean.

Findings table

Every detection becomes one finding with a severity, a human explanation, a MITRE ATT&CK technique and the time window it covers. The screenshot shows the first seven rows; the sample capture produces 22 findings in total.

Findings					
Severity	Device	Finding	Detail	ATT&CK	Window (UTC)
1 critical	cam-garage	2 Contact with known-bad IP	3 121 flow(s) to threat-intel listed address 203.0.113.66	4 T1071 Application Layer Protocol (C2)	5 2026-01-01 11:10:00 2026-01-01 11:39:55
critical	cam-garage	Contact with known-bad IP	1 flow(s) to threat-intel listed address 198.51.100.77	T1071 Application Layer Protocol (C2)	2026-01-01 11:31:00 2026-01-01 11:31:00
high	cam-garage	Suspicious port 4444 (Metasploit default)	121 flow(s) to port 4444; destinations: 203.0.113.66	T1571 Non-Standard Port	2026-01-01 11:10:00 2026-01-01 11:39:55
high	cam-garage	Suspicious port 23 (Telnet)	30 flow(s) to port 23; destinations: 10.50.0.100, 10.50.0.101, 10.50.0.102, 10.50.0.103, 10.50.0.104	T1571 Non-Standard Port	2026-01-01 11:19:00 2026-01-01 11:19:23
high	cam-garage	Suspicious port 2323 (Telnet-alt)	1 flow(s) to port 2323; destinations: 10.50.0.20	T1571 Non-Standard Port	2026-01-01 11:19:31 2026-01-01 11:19:31
high	cam-garage	Periodic beaconing to 203.0.113.66:4444	121 flows, mean interval 15.0s (jitter CV 0.02), avg 310 B/flow	T1071 / T1029 Scheduled Transfer	2026-01-01 11:10:00 2026-01-01 11:39:55
high	cam-garage	Scan-like burst across 10 distinct hosts	10 distinct hosts within 60s window	T1018 Remote System Discovery	2026-01-01 11:18:11 2026-01-01 11:19:05
... 15 more findings not shown					

Figure: Findings table

- Severity.** Critical and high findings come first. Colour and text both carry the severity, so it is readable without colour.
- Finding.** What was detected, e.g. contact with a known-bad IP or periodic beaconing.
- Detail.** The evidence behind it: flow counts, mean beacon interval, jitter, megabytes sent.
- ATT&CK.** The MITRE technique the behaviour maps to, handy for SOC reports.
- Window (UTC).** First and last time the behaviour was seen in the capture.

Live lab capture

In the Docker lab the cameras and the trojan simulator append flow rows to a shared `flows.csv`.

Analyse live lab capture reads that file, using whichever baseline was frozen earlier.

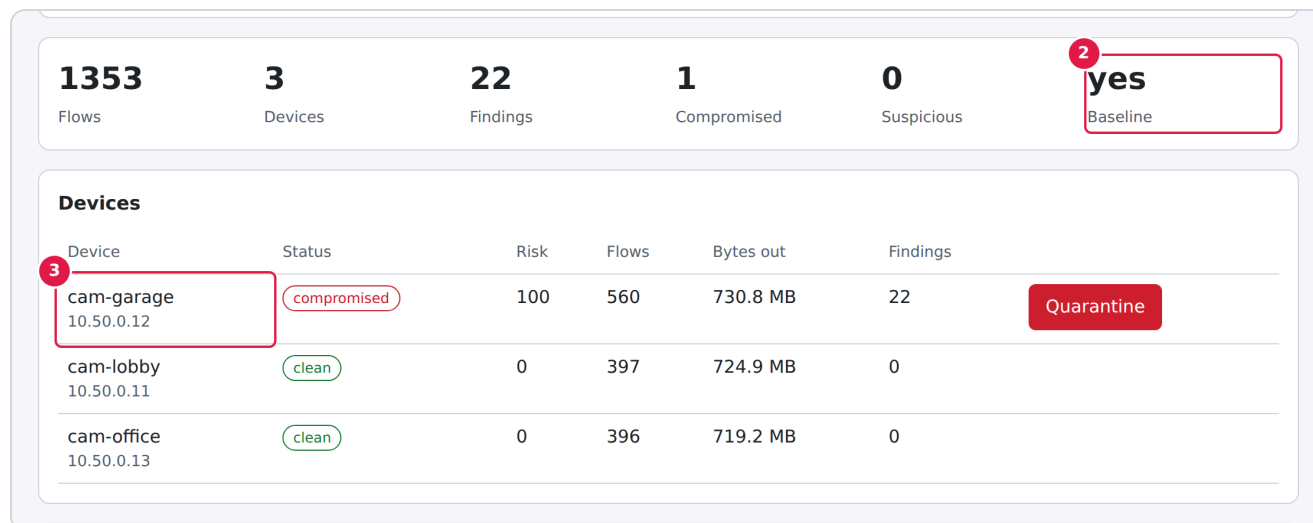


Figure: Live lab capture

- 1 Reads the live `flows.csv` instead of an upload.
- 2 **Baseline: yes.** Confirms the comparison used a known-good profile, which enables the “new destination” and “beacon outside baseline” logic.
- 3 The infected camera is identified by device name and source IP.

A clean capture

Normal camera traffic (RTSP video, NTP, DNS, cloud heartbeat) produces zero findings, even though that traffic is itself periodic. This is what keeps false positives low.

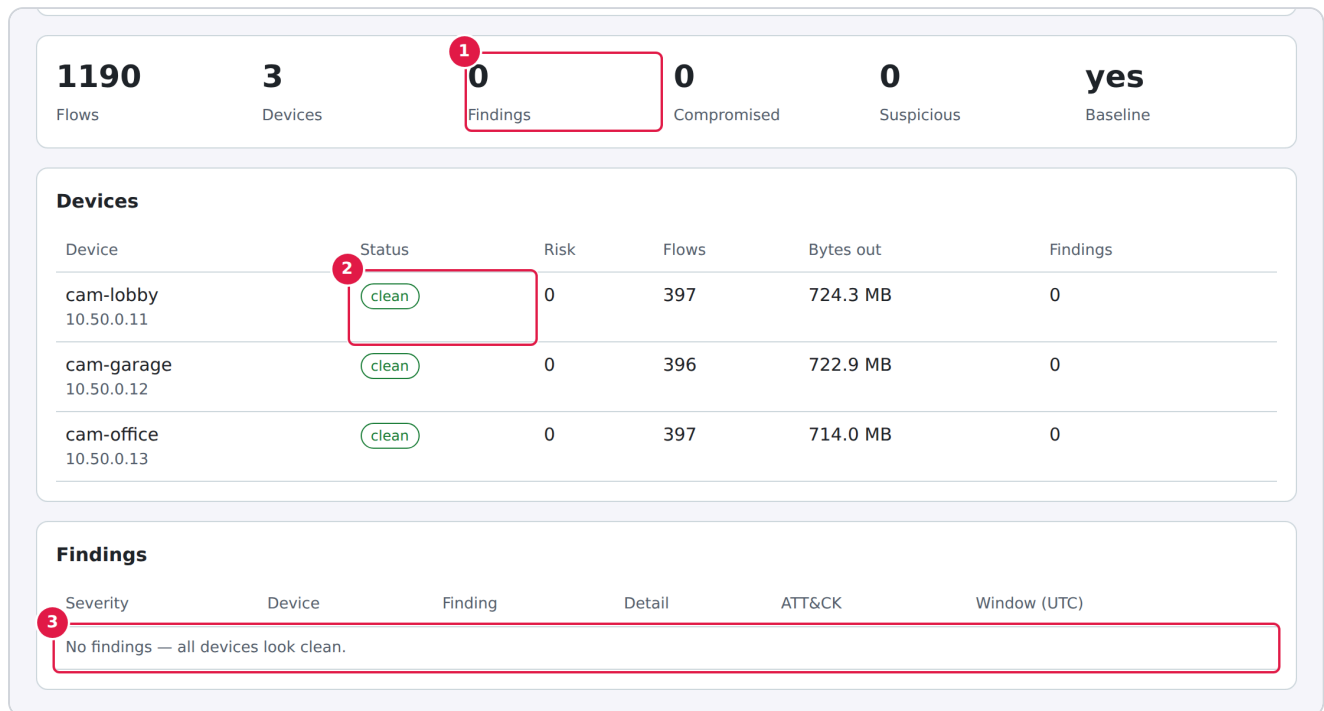


Figure: A clean capture

- 1 0 findings.
- 2 Every device is *clean* with a risk score of 0, so no Quarantine button is offered.
- 3 The findings table states that nothing was found.

Quarantine and release

Pressing **Quarantine** calls `POST /api/quarantine/<device>`. In the lab this writes a marker file that the trojan simulator notices, after which it deletes its persistence files and exits.

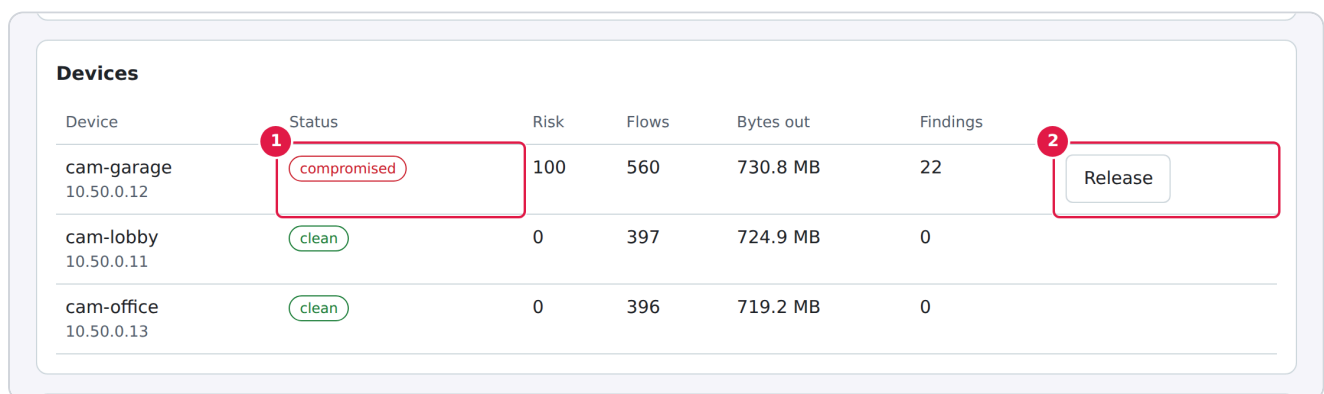


Figure: Quarantine and release

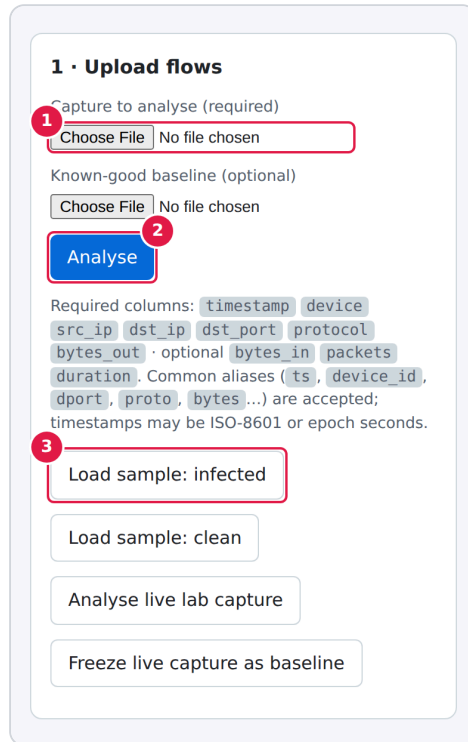
- 1 The device is still reported as compromised: quarantine is a response action, not a verdict change.
- 2 **Release** replaces the Quarantine button and calls `DELETE /api/quarantine/<device>`, which removes the marker.

5. Mobile view

The UI is responsive down to phone width (captured at 390×844 CSS pixels, 2× density). Cards stack, tiles reflow and wide tables scroll inside their own card. The capture script also checks `document.scrollWidth` and reports no page-level horizontal overflow.

Mobile: upload screen

On a phone the layout is a single column. File pickers and buttons wrap, and the page never scrolls sideways.



The image shows a mobile UI for uploading flows. It is titled "1 · Upload flows". The first section, "Capture to analyse (required)", contains a red-bordered button labeled "Choose File" with the text "No file chosen" next to it, marked with a red circle 1. Below this is a section for "Known-good baseline (optional)" with a grey-bordered "Choose File" button and "No file chosen" text. A blue "Analyse" button is marked with a red circle 2. Below the buttons is a list of required columns: timestamp, device, src_ip, dst_ip, dst_port, protocol, bytes_out, and optional bytes_in, packets, duration. It also mentions common aliases (ts, device_id, dport, proto, bytes...) and that timestamps may be ISO-8601 or epoch seconds. The second section, marked with a red circle 3, contains four buttons: "Load sample: infected" (highlighted with a red border), "Load sample: clean", "Analyse live lab capture", and "Freeze live capture as baseline".

Figure: Mobile: upload screen

- 1 The two file pickers stack vertically.
- 2 **Analyse** keeps a full-size tap target.
- 3 The sample, live and baseline buttons wrap onto extra rows.

Mobile: summary and devices

Summary tiles reflow into a two-column grid. The device table sits in a horizontally scrollable container, so only the table scrolls, not the page.

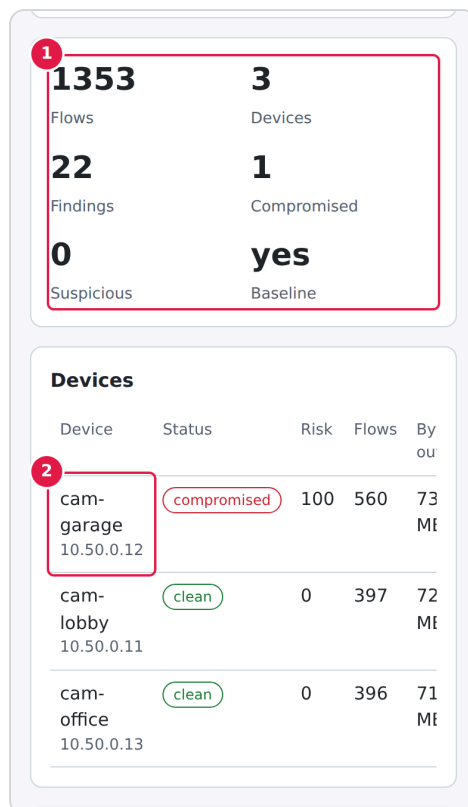
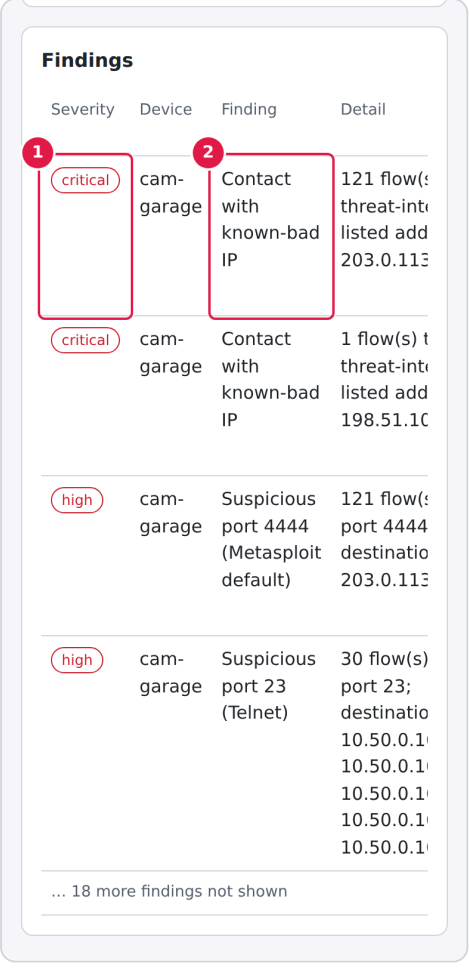


Figure: Mobile: summary and devices

- 1 Tiles reflow to two columns.
- 2 Device name and source IP are in the first column; swipe the table sideways to reach status, risk, bytes out and the Quarantine action. The page itself does not scroll sideways.

Mobile: findings

The findings table also scrolls inside its own card; severity and finding title are visible first.

A mobile application UI mockup showing a 'Findings' table. The table has four columns: Severity, Device, Finding, and Detail. It contains four rows of data. The first two rows are marked as 'critical' and the last two as 'high'. Red callout boxes with numbers 1 and 2 highlight the severity pill and the finding title, respectively. A red box also highlights the first two columns of the first row. A scroll bar is visible on the right side of the table, and a footer indicates that 18 more findings are not shown.

Severity	Device	Finding	Detail
critical	cam-garage	Contact with known-bad IP	121 flow(s) threat-intel listed add 203.0.113
critical	cam-garage	Contact with known-bad IP	1 flow(s) threat-intel listed add 198.51.100
high	cam-garage	Suspicious port 4444 (Metasploit default)	121 flow(s) port 4444 destination 203.0.113
high	cam-garage	Suspicious port 23 (Telnet)	30 flow(s) port 23; destination 10.50.0.10 10.50.0.11 10.50.0.12 10.50.0.13

... 18 more findings not shown

Figure: Mobile: findings

- 1 Severity pill.
- 2 Finding title; swipe left for detail, ATT&CK mapping and time window.

6. Running locally

Option A: detector only (no Docker)

```
cd trojan-lab
pip install -r detector/requirements.txt
LAB_DATA_DIR=/tmp/lab python detector/app.py      # http://localhost:8080
```

Use **Load sample** or upload `sample_data/*.csv` . The live-capture buttons need `$LAB_DATA_DIR/flows.csv` , which only the lab produces.

Option B: full lab with Docker

```
cd trojan-lab
docker compose up --build -d          # cameras, RTSP, C2, detector
# open http://localhost:8080 and click "Freeze live capture as baseline" after ~1 minute
docker compose --profile trojan up -d trojan # infect cam-garage (harmless simulator)
# after ~2 minutes click "Analyse live lab capture", then Quarantine on cam-garage
docker compose logs -f c2              # beacons arrive, then stop
docker compose down -v                  # tear down
```

URL	What
<code>http://localhost:8080</code>	Detector UI
<code>http://localhost:8081</code>	cam-lobby admin page (<code>admin / admin</code>)
<code>rtsp://localhost:8554/cam</code>	Looping test video

Generate your own sample data

```
python trojan-lab/detector/flowgen.py --out trojan-lab/sample_data --seed 7
```

Rebuild this document

```
pip install playwright flask && playwright install chromium # or set CHROMIUM_PATH
make docs              # = capture_screenshots.py then build_pdf.py
```

Environment variables: `LAB_DATA_DIR` (default `/data`), `SAMPLE_DIR` , `PORT` (default 8080), `CHROMIUM_PATH` (docs only).

7. Testing

15 pytest tests run in under a second with no Docker. Run them with `make test` or `python -m pytest trojan-lab/tests`.

File	What it proves
<code>test_engine.py</code>	Clean capture → zero findings (with and without baseline); only cam-garage is compromised in the infected capture; beacon interval is reported near 15 s; column aliases and epoch timestamps parse; missing columns give a clear error; empty files are rejected; irregular traffic is not a beacon.
<code>test_app.py</code>	Upload with a baseline; bad CSV → HTTP 400; missing file → 400; sample analysis and quarantine/release round trip; path-traversal device names rejected.

End-to-end check of the lab logic

The Docker images were not available in the authoring environment, so the simulators were also exercised directly: the fake C2 received beacons from the trojan simulator on localhost, flows landed in `flows.csv`, the engine flagged them, and writing the quarantine marker made the simulator remove its persistence files and exit.

Run `docker compose up --build` once on your machine to validate the container layer (networking, RTSP, volume sharing); it is not covered by the unit tests.

8. Deployment

The lab is a demo. To use the detector on a real network, deploy the same image with real telemetry and put access control in front of it.

Container image

```
docker build -f trojan-lab/detector/Dockerfile -t iot-trojan-detector trojan-lab
docker run -d -p 8080:8080 -v flowdata:/data iot-trojan-detector
```

The image runs gunicorn (2 workers) on port 8080 and reads `/data/flows.csv` for the live buttons.

Feeding real traffic

- Export Zeek `conn.log` or Suricata flow records to the CSV schema above (timestamp, device, src/dst IP, dst port, protocol, bytes out), mapping IP to device name from DHCP or an asset list.
- Write to `/data/flows.csv` on a schedule, or POST files to `/api/analyze` from a cron job or SIEM playbook.
- Freeze a week of known-good traffic as the baseline per device class.

Hardening checklist

Item	Why
Reverse proxy with TLS and authentication (SSO, basic auth, VPN)	The app has no built-in login ; <code>/api/quarantine</code> is a state-changing endpoint.
Replace the marker-file quarantine with a firewall/NAC call	The marker only affects the simulator.
Persist the baseline (database or file)	It is held in process memory and is lost on restart and not shared between the two gunicorn workers.
Tune thresholds and the known-bad list	Constants in <code>engine.py</code> ; load threat-intel feeds into <code>KNOWN_BAD_IPS</code> .
Do not publish the lab ports to untrusted networks	The camera admin page uses <code>admin/admin</code> on purpose.

Known limitation. With `-w 2` each worker keeps its own in-memory baseline, so a baseline uploaded in one request may not be visible to the next. For multi-worker deployments run a single worker or move the baseline to shared storage.

9. Five-minute presentation script

Timed for five minutes at a relaxed pace (about 650 spoken words). Have the app open on the upload page, the sample files ready, and optionally the Docker lab running with `docker compose logs -f c2` in a second window.

Time	On screen	Say / do
0:00–1:00	Cover / README screenshot	"IoT cameras are cheap, always on and rarely patched, which makes them a favourite hiding place for trojans. A compromised camera looks normal on the video side but quietly phones home. I built a lab and a detector that spots that behaviour from network flows alone, so it needs no agent on the device and no malware signature." State the goal: detect, explain, and contain.
1:00–2:00	Screen 1: upload page (open the app)	"Input is just a CSV of flows: timestamp, device, source and destination IP, port, protocol, bytes out. Any Zeek, Suricata or router export maps onto it. Optionally I add a known-good baseline." Point to the schema hint, then the two file pickers. Explain that everything runs in an isolated Docker lab: three simulated cameras, a looping RTSP stream and a harmless trojan stand-in.
2:00–3:00	Click Load sample: clean , then Load sample: infected	"First a clean capture: normal video, NTP, DNS and a cloud heartbeat. Zero findings, even though that traffic is itself periodic, because the engine knows those peers are in the baseline. Now the infected capture." Show the tiles: 1353 flows, 22 findings, one compromised device, and the risk score of 100 on cam-garage.
3:00–4:00	Scroll to the findings table	"Each finding is explained and mapped to MITRE ATT&CK." Walk the top four: contact with a known-bad IP, a beacon every ~15 seconds with tiny jitter, a telnet scan across ten-plus hosts, and an 8 MB transfer to an unknown address. "No single rule is proof, but the combination is a textbook command-and-control pattern." Mention that thresholds are constants at the top of <code>engine.py</code> .
4:00–4:40	Click Quarantine on cam-garage	"Containment is one click. In the lab it drops a marker file; the trojan simulator sees it, deletes its dropper and cron entry, and exits. In production this endpoint would call your firewall or NAC." Show the button turning into Release . If the Docker lab is running, flip to <code>docker compose logs -f c2</code> to show beacons stopping.
4:40–5:00	Close: mobile view + limits	"It also works on a phone for on-call triage. Limits: flow telemetry in the lab is simulated, and the rules are heuristic, so the next steps are real Zeek/Suricata ingestion and learned baselines. Questions?"

If something goes wrong

- No Docker? Use **Load sample** buttons; the story is identical.

- Page shows an error? Say that it is the validation feature, then continue with the sample.
- Short on time? Skip the clean sample and go straight to infected plus quarantine.

Likely questions

Question	Short answer
Why not signatures?	Trojan variants change daily; the behaviour (regular beacons to a new peer, scans, odd ports) does not.
False positives?	Baseline-aware beaconing keeps normal periodic traffic quiet; the clean sample shows zero findings.
Is the trojan real?	No. It reproduces indicators only and cannot leave the isolated lab network.